

Die Vitalzeichen einer Plattform: Wie man mit Observability den Gesundheitszustand von frag.jetzt überwacht

Exposé zur Bachelorarbeit im Studiengang Informatik

Leon Rausch

9. Februar 2026

Zusammenfassung

Warum fühlt sich eine Anwendung zäh an, obwohl alle Server-Statuslampen auf Grün stehen? In der Welt moderner Microservices sind klassische „Geht / Geht nicht“-Alarmer oft nutzlos. Doch wie blickt man hinter die Fassade eines Systems, das aus dutzenden unsichtbaren Bauteilen besteht?

Am Beispiel des Audience Response Systems (ARS) *frag.jetzt* zeigen wir, wie man durch echte Observability Licht in die Black Box bringt. Wir erklären, warum Durchschnittswerte lügen und wie die Four Golden Signals aus dem Google-SRE-Handbuch helfen, Performance-Engpässe zu erkennen, bevor der Nutzer frustriert aufgibt.

1 Notaufnahme für den Code: Warum wir den Puls von frag.jetzt sichtbar machen müssen

Stellen Sie sich folgende Szene vor: Ein vollbesetzter Hörsaal, 500 Studierende starren auf die Leinwand. Der Dozent hat gerade eine interaktive Frage über *frag.jetzt* gestartet. 500 Daumen tippen auf Smartphones. Absenden.

Und dann: **Stille.**

Auf der Leinwand dreht sich der Ladebalken. Sekunden vergehen, die sich wie Minuten anfühlen. Ist das WLAN der Uni zusammengebrochen? Hat sich der Datenbank-Pool erschöpft? Oder wartet das Backend gerade vergeblich auf eine Antwort der externen KI-Schnittstelle?

In der „guten alten Zeit“ monolithischer Software hätte man in eine Log-Datei geschaut und die Antwort gefunden. Doch *frag.jetzt* ist heute ein cloud-natives Nervensystem: Ein Angular-Frontend spricht mit einem NGINX-WebSocket-Gateway, das Anfragen an ein Spring-Boot-Backend weiterreicht; dort wird zur Identitätsprüfung ein externer Dienst (Keycloak) kontaktiert, Nachrichten werden über RabbitMQ verteilt, und schließlich werden komplexe Berechnungen in einem KI-Backend durchgeführt, das wiederum externe LLM-APIs einbindet [1]. Was früher ein einzelner Fehler war, ist nun häufig eine Kettenreaktion.

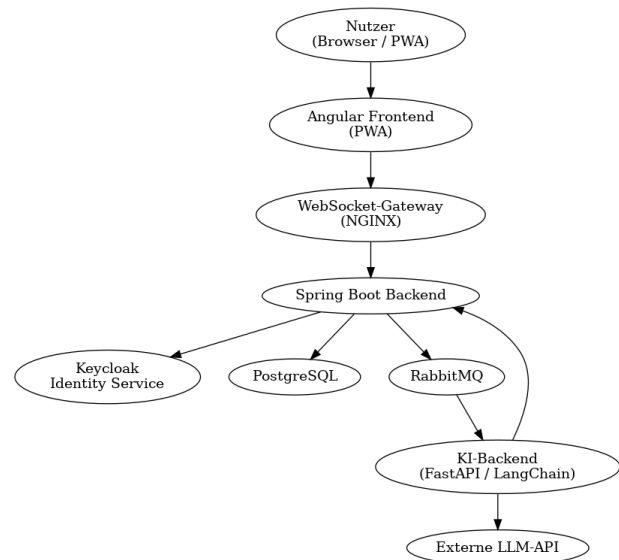


Abbildung 1: Das „Nervensystem“ des Patienten: Der vereinfachte Anfragepfad bei *frag.jetzt*. Eigene Darstellung nach [1].

Wenn hier etwas hakt, ist das System selten komplett „tot“ — es ist „krank“. Es leidet an unsichtbaren Entzündungen: Ein einzelner Service antwortet *beispielhaft* 200 Millisekunden zu spät, und aus dieser Verzögerung wird in der Summe ein Stillstand, der beim Nutzer als spürbare Verzögerung ankommt [2].

Das Problem: Herkömmliches Monitoring sagt uns oft nur: „Der Server ist an.“ oder „der Server ist aus.“ Es sagt nicht, *warum* sich die Anwendung für den Nutzer zäh anfühlt. Hier wechseln wir die Strategie — von reiner Überwachung zu **Observability**. Wir wollen nicht nur wissen, ob das Herz noch schlägt, sondern ob der Patient wirklich stabil ist [3]. Dafür nutzen wir die *Four Golden Signals* als medizinische Vitalwerte.

Dieser Artikel zeigt, wie das ARS *frag.jetzt* in die Diagnose-Röhre geschoben wird, damit aus stummen Daten sprechende Antworten werden — und warum „Ich glaube, es hängt“ keine gültige Fehlerbeschreibung mehr ist.

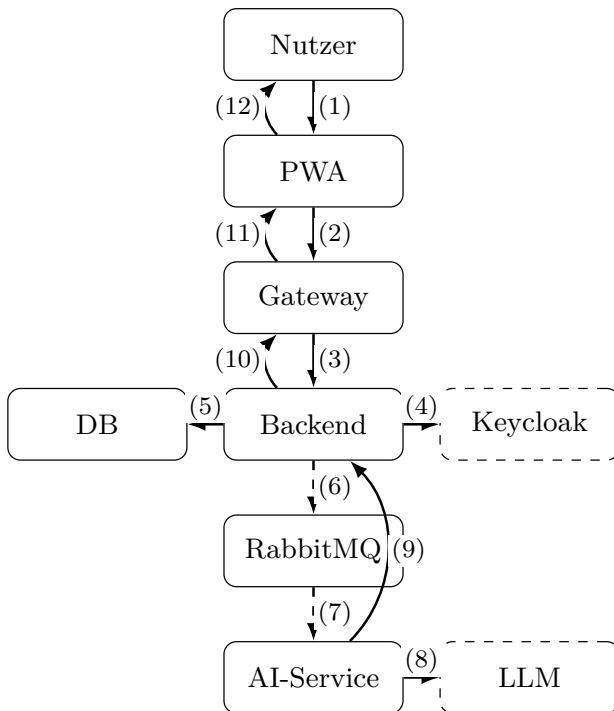


Abbildung 2: Vereinfachter Anfragefluss in der Microservice-Architektur von *frag.jetzt*. Eigene Darstellung nach [1].

2 Vier Golden Signals: Der Gesundheits-Check statt Symptom-Raten

Wenn ein Patient in die Notaufnahme kommt, misst der Arzt nicht „alles“. Er schaut auf wenige Vitalwerte, die schnell Auskunft geben. In der Welt der Google Site Reliability Engineers (SRE) haben sich dafür vier universelle Messkategorien etabliert, die sogenannten *Four Golden Signals* [4]. Für Microservices-Anwendungen wie *frag.jetzt* markieren sie die Grenze zwischen „Ich hoffe, es läuft“ und „Ich weiß, was los ist“.

2.1 Latenz: Die Zeit zwischen Klick und Erlösung

Es ist egal, dass die Datenbank eine Anfrage intern in *beispielhaften* 5 Millisekunden verarbeitet, wenn der Nutzer im Hörsaal über schwaches WLAN erst nach drei Sekunden eine Antwort sieht. Latenz ist die Zeit, die eine Anfrage insgesamt braucht, um bedient zu werden.

Der Trugschluss: Der Durchschnitt beruhigt. Wenn 90 % der Anfragen schnell sind, aber die langsamsten 10 % (die Dozenten-Fragen, die gerade alle sehen) hängen, haben wir ein Problem. Deshalb wird nicht nur der Mittelwert betrachtet, sondern insbesondere die *Tail Latency* — also wie lange die langsamsten Anfragen dauern [3]. Genau dort entsteht Frust, und genau dort verstecken sich die harten Fälle.

2.2 Traffic: Der Moment, in dem alle absenden

Wie viel Last trifft unser System gerade? Bei *frag.jetzt* ist das kein stetiges Rauschen, sondern manchmal ein Tsunami.

Das Szenario: Die Vorlesung beginnt. 500 Studierende öffnen gleichzeitig die App. Der Traffic schießt von 5 auf 500 Anfragen pro Sekunde. Ein plötzlicher Abfall auf Null ist hier übrigens genauso verdächtig wie eine Überlastung: Er bedeutet oft nicht „Feierabend“, sondern „Netzwerkfehler“ oder „Gateway hängt“ [5].

2.3 Fehler: Wenn das Schweigen laut wird

Klassische Fehler sind einfach: Ein HTTP-Code 500 (Internal Server Error) ist ein Schrei nach Hilfe. Doch in verteilten Systemen lauern die gefährlicheren Varianten im Flüsterton.

Der Knackpunkt: Viel bedrohlicher sind die „leisen“ Fehler. Wenn das Backend zwar „Code 200 OK“ antwortet, aber der Inhalt leer ist oder falsche Daten liefert, merkt das klassische Monitoring nichts — der Nutzer aber alles. Observability bedeutet hier, tief genug in die Antwort zu schauen, um auch funktionale Pannen zu erkennen [3]. Das kann bedeuten, dass semantische Checks (z. B. „Antwort enthält mindestens X Elemente“) instrumentiert und als Fehlerrate sichtbar gemacht werden.

2.4 Sättigung: Der Stau vor der Kasse

Sättigung misst, wie „voll“ Ressourcen sind: CPU, Speicher, Thread-Pools, Datenbank-Connection-Pools. Sie ist oft der wichtigste Indikator für das, was als Nächstes passiert [4].

Die Analogie: Die Analogie: Stellen Sie sich einen Supermarkt am Samstagabend vor. Hunderte Kunden wollen bezahlen, aber nur **eine einzige Kasse** ist geöffnet. Solange nur zwei Leute anstehen, läuft der Betrieb. Doch sobald das Kassensband (der Puffer) voll ist, bricht das System zusammen: Der Kassierer (die CPU) arbeitet zwar auf Hochtouren, aber die Schlange staut sich exponentiell bis ins Tiefkühlregal zurück. Sättigung warnt uns also, *bevor* die Wartezeit für alle unerträglich wird — sie ist das ultimative Frühwarnsystem.

2.5 Fazit der Diagnose: Vier Werte, eine Geschichte

Ein einzelner Messwert ist fast immer missverständlich. Hohe Latenz kann an hoher Sättigung liegen — oder an einem Netzwerkfehler. Erst wenn wir diese vier Vitalwerte übereinanderlegen (Korrelation), wird aus Daten echtes Wissen [4]. Und erst dann wird Debugging weniger Ratespiel und mehr Detektivarbeit mit Beweismitteln.

Signal	Typische Metrik	Alarm-Kriterium
Latenz	Dauer der langsamen Anfragen (Spitzenwerte)	Wert steigt dauerhaft an
Traffic	Anfragen pro Sekunde (RPS)	Plötzlicher Einbruch auf Null / extrem viele Anfragen
Fehler	Rate fehlgeschlagener Anfragen	Fehlerrate steigt deutlich
Sättigung	Auslastung (z.B. CPU, Speicher)	Ressource ist fast erschöpft

Tabelle 1: Die Four Golden Signals als Vitalwerte: Was wir messen und wann wir warnen. Nach [sridharan2018distributed, 4].

3 Werkzeug-Tetris: Die Suche nach dem passenden Stack

Wer heute „Observability Tools“ in eine Suchmaschine eingibt, wird von einer Lawine begraben. Es gibt hunderte Dienste, vom teuren Enterprise-SaaS bis zum selbstgebaute Skript. Für ein Open-Source-Projekt wie *frag.jetzt* ist klar: Wir können uns keine Lizenzkosten leisten, aber wir können uns auch keine blinden Flecken leisten.

Wir standen vor der klassischen Frage: Was brauchen wir wirklich — einen Detektiv, einen Wächter oder ein Navigationssystem? Die Antwort lautet: alle drei, aber in der richtigen Dosierung [5].

3.1 Die Wächter: Metriken für den Puls

Unsere erste Wahl fiel auf **Prometheus** und **Grafana**. Warum? Weil Logs teuer sind: Jede Zeile Text zu speichern, kostet Speicherplatz und Rechenleistung. Metriken hingegen sind kompakt, weil sie Zahlenreihen darstellen. Prometheus speichert keine Romane, sondern Messwerte wie „CPU bei 80 %“ oder „50 Anfragen pro Sekunde“, während Grafana diese anschaulich visualisiert [6]. Das ist wie ein EKG: Es zeigt Stress, aber nicht zwangsläufig die Ursache.

3.2 Der Detektiv: Logs (aber nur gezielt)

Für *frag.jetzt* bedeutet das: Der Puls wird überall fühlbar gemacht — an den Schnittstellen, an kritischen Pools, an den Endpunkten, die Nutzer wirklich spüren.

Wenn Prometheus Alarm schlägt, braucht das Team Details. Hier kommt klassisch der **ELK-Stack** (Elasticsearch, Logstash, Kibana) ins Spiel. Aber Vorsicht: In verteilten Systemen explodieren Logs förmlich; „alles loggen“ ist selten nachhaltig [5].

Wir haben gelernt: „Log everything“ ist keine Strategie, sondern ein Weg, die Festplatte zu verstopfen.



Abbildung 3: Das Cockpit für den Notfall: Ein Grafana-Dashboard visualisiert Latenzen und Fehlerraten in Echtzeit. Quelle: [7].

Sinnvoller ist es, Logs dort zu setzen, wo Entscheidungen getroffen werden: bei Fehlermeldungen, bei Zustandswechseln, beim Start der Anwendung — nicht bei jedem Herzschlag [3]. So wird der Detektiv nicht zum Messie.

3.3 Das Navi: Distributed Tracing

Hier wurde es für *frag.jetzt* kritisch. Metriken und Logs sind mächtig, aber ohne Kontext oft isoliert. Sie sagen: „Die Datenbank ist langsam“ und „Das Frontend wartet“. Aber sie beantworten nicht: *Worauf* wartet das Frontend eigentlich, und *welcher* Call hat den Stau ausgelöst?

In einer Microservice-Architektur wie unserer verliert man ohne **Tracing** den Faden [8]. Deshalb werden Tracing-Tools wie **Jaeger** eingesetzt, um den Weg einer einzelnen Anfrage über Gateway, Spring Boot und KI-Backend nachzuzeichnen. Erst Tracing macht aus einzelnen Bauteilen eine zusammenhängende Geschichte [8].

3.4 Der Notruf: Intelligentes Alerting

Metriken, Logs und Traces sind das Nervensystem, aber das Alerting ist der Schmerzreiz, der zum Handeln zwingt. In der Praxis führt „stumpfes“ Alerting (z. B. einfache statische Schwellenwerte wie „CPU über 90 %“) jedoch oft zu einer Flut irrelevanter Benachrichtigungen, die echte Notfälle verdecken.

Moderne Ansätze setzen daher auf **Context Aware Alerting** und **AI-Powered Alerting**, um das Rauschen drastisch zu minimieren (*Noise Reduction*) [9]. Statt bei jedem kurzen Last-Spike sofort Panik zu verbreiten, analysieren intelligente Algorithmen den

Kontext: Ist die hohe Last zu dieser Uhrzeit typisch? Korreliert der Anstieg mit einem geplanten Hintergrundprozess?

Durch diese kontextbezogene Filterung unterscheidet das System zwischen einem harmlosen „Schluckauf“ und einem ernsthaften Vorfall. Der digitale Wecker wird so vom dauerläutenden Störsignal zu einem verlässlichen Notrufsystem, das nur dann Alarm schlägt, wenn menschliches Eingreifen tatsächlich notwendig ist [9].

3.5 Die Geheimwaffe: OpenTelemetry gegen blinde Flecken

Das größte Risiko bei der Tool-Auswahl ist der „Vendor Lock-in“. Was, wenn morgen von Jaeger zu Tempo gewechselt wird? Oder von Prometheus zu einer Cloud-Lösung? Eine Observability-Strategie sollte nicht auf Werkzeug-Markennamen beruhen, sondern auf austauschbaren Standards.

Hier setzt **OpenTelemetry (OTel)** an: als universeller Adapter, der Instrumentierung vom Backend bis zum Gateway neutral hält und Daten an die Tools der Wahl verteilt [8]. Statt Code mit einer Prometheus-spezifischen Bibliothek zu „verdrahten“, wird alles über OTel gesammelt und exportiert. Das reduziert Abhängigkeiten und erleichtert spätere Entscheidungen [8].

Im Krankenhaus	In der IT	Die zentrale Frage
Pulsmesser (EKG)	Metriken (Prometheus)	Ist das System gesund?
Krankenakte	Logs (ELK, Loki)	Was ist schiefgelaufen?
Kontrastmittel	Tracing (Jaeger)	Wo ist der Engpass?
Notruf	Alerting	Muss jemand geweckt werden?

Tabelle 2: Vom Symptom zur Diagnose: Die medizinischen Werkzeuge und ihre digitalen Gegenstücke.

3.6 Ein nüchterner Blick: Die Grenzen der Beobachtbarkeit

So mächtig Observability in der Praxis ist, sie ist weder kostenlos noch vollständig. Jede zusätzliche Metrik, jeder Trace und jede Logzeile erzeugt Daten, die gesammelt, gespeichert, ausgewertet und verstanden werden müssen. Damit wird Observability selbst zu einer ingenieurtechnischen Disziplin, die Pflege, Erfahrung und bewusste Reduktion erfordert [10][5].

Hinzu kommt: Ein System ist immer nur so beobachtbar wie seine Instrumentierung. Was nicht gemessen wird, bleibt unsichtbar – selbst mit dem besten Werkzeug-Stack [2]. Observability verkürzt die Zeit bis zum Verständnis eines Problems, sie verhindert jedoch weder Ausfälle noch komplexe Wechselwirkungen in verteilten Systemen [10].

Gerade deshalb ist Zurückhaltung entscheidend. Ziel ist nicht maximale Datensammlung, sondern maximale Entscheidungsfähigkeit. Oder anders gesagt: Ein gutes Observability-System weiß nicht alles – aber genau das Richtige.

4 Schluss mit dem digitalen Wecker: Warum Stille Gold ist

Hand aufs Herz: Nichts ruiniert die Moral eines Teams schneller als ein Telefon, das nachts um 3 Uhr vibriert, nur um zu melden: „CPU-Auslastung bei 85 %“ — und fünf Minuten später springt der Wert von selbst wieder auf „OK“.

Dieses Phänomen hat einen Namen: **Alarmmüdigkeit** (Alert Fatigue) [11]. Wenn alles ein Notfall ist, ist nichts mehr ein Notfall. Gerade in vielen frühen Betriebsphasen verteilter Systeme entsteht der Impuls, auf jedes Signal zu lauschen. Ein Service antwortet nicht sofort? Alarm. Die Datenbank schreibt viele Logs? Alarm. Mit der Zeit stumpft das Team ab – Benachrichtigungen werden ausgeblendet wie ein dauerhaftes Hintergrundgeräusch. Und ausgerechnet im entscheidenden Moment bleibt der echte Notruf ungehört [11].

Also muss das Alarm-Konzept radikal umgestellt werden — weg von „Rauschen“, hin zu „Handlung“. Drei Regeln können dabei als Leitplanken dienen, weil sie Alarmer zu Entscheidungen machen und nicht zu Nervenkrieg [9][11]:

1. **Weck mich nur, wenn der Nutzer leidet.** Ein hoher CPU-Wert ist kein Grund zur Panik, solange die Antworten schnell kommen. Alarmiert wird auf *Symptome*, nicht auf bloße Ursachen [9]. *Schlecht:* „Server Node 4 hat hohe Last.“ *Gut:* „Die Spitzenlatenz beim Absenden von Fragen ist über 2 Sekunden gestiegen“ [4].
2. **Die „Luft-anhalten“-Strategie.** Netzwerke haben Schluckauf. Das ist normal. Statt bei jedem verlorenen Paket sofort „Feuer!“ zu schreien, wird zeitlich geglättet. Besteht das Problem länger als eine definierte Dauer (z. B. zwei Minuten), wird die Nachricht erst dann aktiv. Das eliminiert nervöse Fehlalarme („Flapping“) fast vollständig [11].
3. **Kein Alarm ohne Spielbuch.** Ein Alarm, der nur sagt „Es ist kaputt“, ist wertlos. Er erzeugt Stress, aber keine Lösung. Deshalb werden **Runbooks**

direkt mit Alerts verknüpft: Wenn der „Datenbank-Pool erschöpft“-Alarm kommt, liefert er die Anleitung gleich mit: „(1) Offene Verbindungen prüfen, (2) Service X neu starten, (3) Lastspitze korrelieren“ [12][11].

4.1 Ein kurzer Einschub: Warum das mehr ist als Tool-Kosmetik

Ein gutes Observability-System ist wie ein guter Rauchmelder: Es schweigt die meiste Zeit. Aber wenn es sich meldet, dann schnappt man sich den Feuerlöscher — und nicht die Ohropax. Diese Logik wird nicht durch „mehr Daten“ erreicht, sondern durch *bessere* Daten, durch Korrelation und durch Alarme, die Handlung erzwingen statt Ablenkung produzieren [12] [11].

5 Fazit: Wenn aus Stille Vertrauen wird

Kehren wir zurück in Hörsaal 1. Wieder zücken 500 Studierende ihre Smartphones. Wieder stellt der Dozent eine Frage über *frag.jetzt*. Und wieder gibt es im Hintergrund vielleicht ein technisches Problem — eine Datenbankabfrage dauert *beispielhaft* 200 Millisekunden zu lang.

Der Unterschied zu früher? Niemand merkt es.

Bevor der Ladebalken auf der Leinwand überhaupt ins Stocken gerät, hat der Observability-Stack das Symptom („Latenz steigt am Backend“) bereits erfasst. Prometheus hat den Trend erkannt, noch bevor ein Grenzwert gerissen wurde [6]. Ein Blick in die Tracing-Ansicht zeigt: „Aha, der Connection-Pool zur Datenbank läuft voll“ [8]. Und das Runbook liefert die Lösung, noch bevor der erste Nutzer frustriert die App schließen kann [12].

Das ist der Kern von Observability: Es geht nicht um bunte Dashboards an der Wand, die beeindruckend aussehen. Es geht darum, auch solche Fehler beherrschbar zu machen, mit denen niemand gerechnet hat – indem das System selbst Hinweise auf Ursachen und Zusammenhänge liefert [3]. Fehler in verteilten Systemen werden nicht vollständig verhindert werden können — Systeme *werden* ausfallen [4]. Aber wir können entscheiden, wie schnell wir verstehen, was passiert, und wie schnell wir wieder stabil sind.

5.1 Der nächste Schritt: Von „Best Effort“ zu Service Level Objectives (SLOs)

Sind wir damit am Ziel? Nein. Observability ist kein Projekt, sondern ein fortlaufender Prozess. Zunächst wird sichtbar gemacht, wie sich ein System tatsächlich verhält. Der nächste Schritt ist die Definition dessen, was *sein darf*. Mit **Service Level Objectives (SLOs)** wird festgelegt, welche Nutzererfahrung als „gut genug“ gilt, etwa: „99 % aller Fragen müssen in

unter 500 ms beantwortet sein“ [4]. Das zwingt dazu, Technik nicht nur technisch zu betrachten, sondern vom Nutzer her zu denken. Und es schafft eine klare Konsequenz: Wenn das Ziel verfehlt wird, werden neue Features gestoppt und Stabilität priorisiert [3].

Am Ende ist *frag.jetzt* mehr als Code. Es ist die Interaktion zwischen Menschen. Und die Aufgabe einer Observability-Strategie besteht darin, sicherzustellen, dass die Technik dieser Interaktion niemals im Weg steht.

Quellen und weiterführende Literatur

- [1] TH Mittelhessen und arsnova. *frag.jetzt – Open-Source Q&A-Plattform*. 2025. URL: <https://gitlab.arsnova.eu/arsnova/frag.jetzt/-/blob/staging/README.md> (besucht am 07.02.2026).
- [2] S. Fischer u. a. „An Overview of Microservice-Based Systems Used for Evaluation in Testing and Monitoring: A Systematic Mapping Study“. In: (2024). DOI: 10.1145/3644032.3644445.
- [3] Olumese Anthony Abieba u. a. „Advanced Observability and Monitoring Practices for Enhancing Production Environment Reliability“. In: (2025). URL: <https://worldscientificnews.com/wp-content/uploads/2025/06/WSN-204-2025-270-296.pdf> (besucht am 07.02.2026).
- [4] Rob Ewaschuk. *Monitoring Distributed Systems*. <https://sre.google/sre-book/monitoring-distributed-systems/>. Kapitel 6 in *Site Reliability Engineering*, herausgegeben von Betssy Beyer, Google. 2017. (Besucht am 07.02.2026).
- [5] Luca Giamattei u. a. „Monitoring tools for DevOps and microservices: A systematic grey literature review“. In: *Journal of Systems and Software* 208 (2024), S. 111906. DOI: 10.1016/j.jss.2023.111906.
- [6] A. Banerjee und R. Singh. „A Comparative Analysis of System Monitoring Architecture: Evaluating Prometheus, ELK Stack, and Custom Dashboards for Performance and Scalability“. In: *International Journal of Computer Applications* (2024). DOI: 10.21275/SR25917095133.
- [7] Grafana Labs. *Grafana Dashboards Overview*. 2026. URL: <https://grafana.com/docs/grafana/latest/fundamentals/dashboards-overview/> (besucht am 07.02.2026).
- [8] Carlos Albuquerque und Filipe F Correia. „Tracing and Metrics Design Patterns for Monitoring Cloud-native Applications“. In: *arXiv preprint arXiv:2510.02991* (2025). URL: <https://arxiv.org/abs/2510.02991> (besucht am 07.02.2026).

-
- [9] Kowshik Sakinala. „Monitoring and Observability for Cloud-Native Applications“. In: *Journal of Computer Science and Technology Studies* 7.8 (2025), S. 101–115. URL: <https://al-kindipublishers.org/index.php/jcsts/article/view/10459> (besucht am 07.02.2026).
 - [10] U. Faseeha u. a. „Observability in Microservices: An In-Depth Exploration of Frameworks, Challenges, and Deployment Paradigms“. In: *IEEE Access* 13 (2025), S. 72011–72035. DOI: 10.1109/ACCESS.2025.3562125.
 - [11] Fatemeh Jalalvand u. a. „Alert prioritisation in security operations centres: A systematic survey on criteria and methods“. In: *ACM Computing Surveys* 57.2 (2024), S. 1–36. DOI: 10.1145/3695462.
 - [12] Harish Govinda Gowda. „From Alerts to Automation: Building Self-Healing SRE Frameworks with Runbooks and Intelligent Triggers“. In: *International Journal of Science, Engineering and Technology* 10.6 (2022), S. 1–9. ISSN: 2348-4098. URL: https://www.researchgate.net/profile/Abubakr-Hameed/publication/393712529_From_Alerts_to_Automation_Building_Self-Healing_SRE_Frameworks_with_Runbooks_and_Intelligent_Triggers/links/6876a00add6b84447df8ca86/From-Alerts-to-Automation-Building-Self-Healing-SRE-Frameworks-with-Runbooks-and-Intelligent-Triggers.pdf (besucht am 07.02.2026).

Hinweis zur KI-Nutzung: Bei der Erstellung dieses Exposés wurden KI-Tools (ChatGPT 5.2, Google Gemini Pro 3, Consensus, Elicit) zur Unterstützung bei Recherche und Formulierung eingesetzt. Die inhaltliche Verantwortung liegt beim Autor.